

ジャンルキット

ニッチ向け拡張と横断基盤機能を提供します。

MPTK 出力ルーティング: キットの `outputDeviceId` が空の場合、`outputPreset` (`MidiOutputRoutingPreset`) で MPTK 仮想出力 (`mptk:internal`) 等へ共通ルーティングできます。同梱プリセット:

`Assets/MIDI/Scripts/Integrations/MPTK/Presets/MptkVirtualOutput.preset.asset`。詳細は [Maestro / MPTK 統合 — サンプルシーン / 出力プリセット / エディタ試聴](#)。

キット / 機能	名前空間	Assembly Definition	スクリプト定義シンボル
ネットワーク MIDI 同期	<code>jp.kshoji.unity.midi.net</code>	<code>jp.kshoji.midi.net</code>	<code>FEATURE_MIDI_NETWORK</code>
MidiClockSync	<code>jp.kshoji.unity.midi</code>	<code>jp.kshoji.midi</code> (コア)	不要
和音・スケール判定	<code>jp.kshoji.unity.midi / .util</code>	<code>jp.kshoji.midi</code> (コア)	不要
Input System ブリッジ	<code>jp.kshoji.unity.midi.integrations.inputsystem</code>	<code>jp.kshoji.midi.inputsystem</code>	<code>FEATURE_INPUT_SYSTEM</code>
Foundation	<code>jp.kshoji.unity.midi.foundation</code>	<code>jp.kshoji.midi.foundation</code>	不要

配置:

キット	配置
ネットワーク	<code>Assets/MIDI/Scripts/Integrations/Networking/</code>
Clock / 和音判定 (コア)	<code>Assets/MIDI/Scripts/Gameplay/</code> (<code>Theory/</code> 含む)
Input System	<code>Assets/MIDI/Scripts/Integrations/InputSystem/</code>
Foundation	<code>Assets/MIDI/Scripts/Foundation/</code>
Foundation UI	<code>Assets/MIDI/UI/Foundation/</code>
サンプル	<code>Assets/MIDI/Samples/Gameplay/</code> / <code>Assets/MIDI/Samples/Integrations/Networking/</code> / <code>Assets/MIDI/Samples/Integrations/InputSystem/</code> / <code>Assets/MIDI/Samples/Foundation/</code>

キット間の依存関係

キット / 機能	主な依存コンポーネント
ネットワーク MIDI 同期	<code>SmfPlayer</code> , <code>MidiManager</code>
Input System ブリッジ	<code>MidiManager</code>
Timeline 統合	<code>SmfPlayer</code> , <code>TempoMapExtractor</code> , <code>MidiRecorder</code>
Animator 統合	<code>MidiInputRouter</code> , <code>MidiCcSmoother</code> , <code>MidiNoteTracker</code>

ネットワーク MIDI 同期

LAN 上で MIDI イベントと SMF 再生位置を同期します。UDP ベースの軽量トランスポートを内蔵し、Netcode / Mirror 等の追加パッケージは不要です。

前提条件

Project Settings にスクリプト定義シンボルを追加:

- FEATURE_MIDI_NETWORK

項目	値
Assembly Definition	jp.kshoji.midi.net
設定パス	Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

主要コンポーネント

コンポーネント	役割
MidiNetworkHub	ホスト。MIDI イベント配信
MidiNetworkClient	受信イベントを仮想デバイスへ注入
MidiNetworkMessage	シリアライズ可能 DTO
MidiPlaybackSync	SmfPlayer 再生位置同期 (broadcastInterval 既定 0.1s、seekThreshold 既定 0.05s)
MidiSessionDiscovery	LAN セッション UDP アドバタイズ (payload の hubPort)
MidiSessionDiscoveryListener	LAN セッション UDP 受信 (OnSessionAdvertisement)
MidiLatencyCompensation	RTT ヘルパー (UDP Ping/Pong → estimatedRttMs)

同期モード

モード	用途
Broadcast	ホスト入力を全クライアントへ配信
Merge	全クライアント入力をホストで合流
Playback	再生位置のみ同期

オプション: Mirror / Netcode / WNet2 ブリッジ

UDP の MidiNetworkHub / MidiNetworkClient に追加パッケージは不要です。オプションのブリッジは同じ MidiNetworkMessage / MidiNetworkMessageCodec を、既に使っているゲーム用トランスポートへ載せます。パッケージは 本リポジトリに同梱しません。

シンボル	アセンブリ	コンポーネント	パッケージ / ゲート
FEATURE_MIRROR (+ FEATURE_MIDI_NETWORK + MIRROR)	jp.kshoji.midi.net.mirror	MidiMirrorBridge	Mirrorを導入。MIRROR は Mirror 側定義
FEATURE_NETCODE (+ FEATURE_MIDI_NETWORK + MIDI_HAS_NETCODE)	jp.kshoji.midi.net.netcode	MidiNetcodeBridge	UPM com.unity.netcode.gameobjects (versionDefines → MIDI_HAS_NETCODE)
FEATURE_WSNET2 (+ FEATURE_MIDI_NETWORK + MIDI_HAS_WSNET2)	jp.kshoji.midi.net.wsnet2	MidiWsnet2Bridge	WSNet2 クライアント + WSNet2.Runtime.asmdef (Optional ヘルパー)。Editor が MIDI_HAS_WSNET2 を同期。Lobby/Game サーバは別途起動

有効化の流れ:

1. FEATURE_MIDI_NETWORK と上記ブリッジ用シンボルを追加
2. 対応フレームワークを導入 (配布ブランチに Mirror / NGO / WSNet2 を残さない)
3. Host/Master の hub と peer の playbackSync を配線 ([Networking 統合](#))
4. Assets/MIDI/Samples/Integrations/Networking/Scenes/ のサンプルを開く

サンプル

シーン	前提
.../Networking/Scenes/MidiNetworkJamSampleScene.unity	FEATURE_MIDI_NETWORK
.../Networking/Scenes/MidiMirrorNetworkSampleScene.unity	Mirror + FEATURE_MIRROR
.../Networking/Scenes/MidiNetcodeNetworkSampleScene.unity	NGO + FEATURE_NETCODE
.../Networking/Scenes/MidiWsnet2NetworkSampleScene.unity	WSNet2 + サーバ + FEATURE_WSNET2

横断基盤

外部 MIDI Clock 同期、和音・スケール判定、Input System ブリッジを提供します。詳細は [ゲームプレイ向けコンポーネント](#) も参照してください。

MidiClockSync

外部 MIDI Clock（Timing Clock / Start / Stop / Continue）に同期し、BPM・拍位置・小節位置を提供します。

プロパティ / API	説明
<code>pulsesPerQuarterNote</code>	1 拍あたりの Clock パルス数（MIDI 標準 24）
<code>beatsPerBar</code>	小節あたりの拍数（ <code>onBar</code> 用、既定 4）
<code>deviceIdFilter</code>	空 = 全デバイス
<code>EstimatedBpm</code>	直近パルスから推定した BPM
<code>IsBpmStable</code>	BPM 推定が安定したか
<code>onBeat</code> / <code>onBar</code>	拍 / 小節境界イベント
<code>onStarted</code> / <code>onStopped</code>	Start / Stop 受信イベント

`SmfPlayerClockAdapter` は `SmfPlayerClockMode` で動作を切り替えます:

モード	動作
<code>Follow</code>	外部 Clock の推定 BPM を <code>SmfPlayer.tempoBpm</code> に反映
<code>Step</code>	外部 Clock の各拍で SMF 再生位置を 1 拍進める
<code>Free</code>	外部 Clock を無視

制限: Song Position Pointer (SPP)、MTC、Ableton Link は未対応です。

和音・スケール判定

コンポーネント / ユーティリティ	役割
<code>ChordRecognition</code>	押下ノート集合から和音名（ <code>C</code> 、 <code>Cm7</code> 等）を推定
<code>ScaleUtility</code>	<code>MidiScaleUtility</code> を拡張したスケール所属判定
<code>MidiChordDetector</code>	<code>MidiNoteTracker</code> 連携、和音変化 / スケール外検出 / 目標和音クイズ

Input System ブリッジ（オプション）

コンポーネント	役割
<code>MidiSyntheticDevice</code>	Synthetic Input Device（128 ノート + 128 CC + チャンネル専用軸）
<code>MidiInputSystemBridge</code>	MIDI → Input System 状態注入。SysEx / Raw UMP は <code>onMessage</code>
<code>InputSystemToMidiBridge</code>	Input Action → MIDI 送信
<code>MidiInputSystemMapping</code>	Action ↔ MIDI 条件の ScriptableObject

前提: Package `com.unity.inputsystem`、シンボル `FEATURE_INPUT_SYSTEM`。
セットアップと Synthetic Device レイアウトの詳細は [Unity エコシステム統合 — Input System](#) を参照してください。

`Assets > Create > MIDI > Input System > Mapping` で `MidiInputSystemMapping` を作成します。`MidiActionBinding` の主なフィールド:

フィールド	説明
<code>actionName</code>	<code>.inputactions</code> 内の Action 名
<code>messageType</code>	NoteOn / ControlChange / PitchWheel / ProgramChange 等
<code>group</code>	0–15、-1 = 全 group
<code>channel</code>	0–15、-1 = 全チャンネル
<code>controllerOrNote</code>	CC 番号またはノート番号
<code>valueFilter</code>	値フィルタ、-1 = すべて
<code>targetControlIndex</code>	CC フォールバック先 (-1 = <code>controllerOrNote</code>)
<code>preferDedicatedControl</code>	専用軸 (pitch / program / channelPressure / systemPulse) を優先
<code>invertAxis</code>	正規化軸の反転

サンプル

シーン	説明
<code>Assets/MIDI/Samples/Gameplay/Scenes/MidiClockSyncSampleScene.unity</code>	Clock 注入 / BPM 推定 / <code>SmfPlayerClockAdapter</code>
<code>Assets/MIDI/Samples/Gameplay/Scenes/ChordScaleSampleScene.unity</code>	和音認識・スケールクイズ
<code>Assets/MIDI/Samples/Gameplay/Scenes/ChordPuzzleSampleScene.unity</code>	目標和音クイズ (<code>MidiChordDetector</code>)
<code>Assets/MIDI/Samples/Gameplay/Scenes/ScalePracticeSampleScene.unity</code>	スケール練習
<code>Assets/MIDI/Samples/Integrations/InputSystem/Scenes/InputSystemBridgeSampleScene.unity</code>	Synthetic Device デモ (<code>FEATURE_INPUT_SYSTEM</code> 必須)
<code>Assets/MIDI/Samples/Foundation/Scenes/FoundationSampleScene.unity</code>	デバイス選択・レイテンシ校正・ Foundation UI シェル

関連ドキュメント

- [サンプル](#) — [キット サンプルシーソー](#) 覧
- [ビルド後の処理](#) — [ネットワーク オプションキット](#)
- [ビルド後の処理](#) — [横断基盤 オプションキット](#)
- [SMF ツール](#) — `SmfPlayer` / `TempoMapExtractor`
- [ゲームプレイ向けコンポーネント](#) — `MidiClockSync` / `MidiCcSmoother` / [和音判定](#)